Clase 6

@September 14, 2026

Introducción

En esta sesión hicimos una pausa para resolver dudas sobre el lenguaje C++, la arquitectura de la API de Windows (Win32), la administración de memoria y los fundamentos del modelo de componentes COM de DirectX (Realmente el profe nos obligó a preguntarle cosas porque ya llevaba tiempo pidiéndonos que le anotemos preguntas y nadie hizo nada). Adicionalmente, se abordó la relevancia profesional del rol de **Rendering Engineer / Graphics Programmer** en la industria del software y videojuegos.

Desarrollo

¡Resolución de Dudas!

`const` vs `constexpr`

- `const`: Promete inmutabilidad ser solo de lectura. El valor se puede calcular tanto en tiempo de compilación como en tiempo de ejecución.
- `constexpr`: Obliga a que la expresión o variable sea evaluada estrictamente durante el **tiempo de compilación**. El valor debe ser conocido antes de generar el archivo ejecutable.

> Todo `constexpr` es implícitamente `const`, pero no todo `const` es `constexpr`.

Tipos de Datos y Alias de Win32

`DWORD` (*Double Word*)

Es un alias (*typedef*) de la API de Windows que representa un entero sin signo de 32 bits (`unsigned long`). Su nombre proviene de la arquitectura x86, donde una "palabra" (*Word*) equivale a 16 bits.

`unsigned int`

Entero sin signo. Representa exclusivamente números enteros no negativos. Al no reservar un bit para indicar el signo, permite duplicar el rango positivo de valores representables en memoria.

unsigned int	**4,294,967,295**
signed int	**| 2,147,483,647 |**

`static_cast`

Es un operador de conversión de tipos de dato que se verifica en tiempo de compilación.



Casteo Explícito: Significa que **el programador escribe** formalmente la instrucción de conversión en lugar de permitir que C++ realice una conversión implícita. Al hacerlo explícito, el desarrollador demuestra intención directa ante el compilador y asume la responsabilidad de posibles pérdidas de precisión.

`if (FAILED(result))`

En DirectX y Win32, la mayoría de los métodos devuelven un tipo numérico llamado `HRESULT`. Los códigos `HRESULT` representan éxito con valores mayores o iguales a cero (como `S_OK = 0`) y errores/fallas mediante **números enteros negativos.** La macro `FAILED(result)` evalúa el bit de signo de `result`: si es verdadero (`true`), confirma que la operación falló.

Inclusión de Librería `<sstream>`

En la clase agregamos la librería `<sstream>`. Ella arrastra plantillas de código pesadas e infraestructura compleja de entrada/salida de la librería estándar de C++. Incluirla en archivos de cabecera (`.h`) infla los tiempos de compilación y

el tamaño del binario, y contamina las dependencias. La estrategia óptima en arquitectura de motores gráficos es mantener las cabeceras limpias (utilizando *forward declarations* o el patrón PImpl) y recurrir a métodos de formateo de texto ligeros o restringidos únicamente a los archivos de implementación (`.cpp`).

Identificador COM `__uuidof`

`__uuidof` es un operador nativo del compilador Microsoft Visual C++ que extrae el identificador único global asociado a una interfaz COM de DirectX en tiempo de compilación. Se utiliza para que la GPU y el sistema COM reconozcan exactamente qué tipo de recurso se está solicitando o manipulando en memoria.

Asignación de Memoria: Estática vs Dinámica

Característica	Memoria Estática	Memoria Dinámica
Ubicación	Pila (*Stack*) o área global/estática.	Montón (*Heap*).
Asignación	Definida durante la compilación o inicio del ámbito (*scope*).	En tiempo de ejecución mediante `new`, `malloc` o APIs (p. ej. `CreateBuffer`).
Vida Útil	Automática; se destruye al salir del bloque de código.	Manual; persiste hasta ser liberada explícitamente (`delete`, `free` o `Release()`).
Rendimiento	Muy rápido (desplazamiento directo de puntero).	Requiere búsqueda de bloques libres en memoria y gestión de fragmentación.

Análisis de Industria: Perfil de *Video Game Rendering Engineer*

El puesto de **Graphics/Rendering Engineer** es uno de los roles técnicamente más competitivos y mejor remunerados en el desarrollo de software y motores de videojuegos. Esto se debe a la estricta exigencia matemática, la necesidad de dominar el hardware a bajo nivel y la escasez de especialistas capacitados para optimizar el rendimiento en tiempo real.

Fundamentos Técnicos

- C++17/20 Moderno

Herramientas y Perfilado

- RenderDoc, PIX, Nsight

- Arquitectura de Memoria
- APIs:DX11/12, Vulkan
- Algebra Lineal y Cálculo
- Algoritmos de Iluminación
- Shaders (HLSL, GLSL)
- Compute Shaders & PBR

Requisitos Principales en Vacantes de la Industria (AAA / Engine Tech)

Dominio Avanzado de C++

Manejo manual de memoria, optimización para Caché de la CPU (*Data-Oriented Design*), punteros inteligentes y metaprogramación.

APIs Gráficas de Bajo Nivel

Experiencia demostrable en **DirectX 11/12**, **Vulkan** o **Metal**, comprendiendo la sincronización de comandos entre CPU y GPU.

Programación de Shaders

Dominio de HLSL o GLSL para implementar técnicas como *Physically Based Rendering* (PBR), sombreado, cálculo de sombreadores (*Compute Shaders*) y trazado de rayos (*Ray Tracing*).

Bases Matemáticas Sólidas

Uso fluido de vectores, matrices 4×4, cuaterniones, transformaciones espaciales y proyecciones de perspectiva.

Perfilamiento y Depuración de GPU

Uso de herramientas de análisis como **RenderDoc**, **Microsoft PIX** o **NVIDIA Nsight** para identificar cuellos de botella en la canalización de renderizado.

Conclusión

Siempre son refrescantes estas clases donde el profe se va de tema y nos habla de como se vive de estas carreras. La economía y el sistema capitalista ya va de bajada, pero la verdad me parece muy interesante que una

complejidad tan densa se albergue en estas disciplinas de graphic y mucho más interesante que haya gente que lo valora y trabaja en ellas.

Glosario

HRESULT

Tipo de dato entero de 32 bits devuelto por funciones de Win32 y COM que indica el resultado de éxito o el tipo de error de una llamada de función.

PImpl (*Pointer to Implementation*)

Patrón de diseño en C++ que oculta los detalles privados de implementación de una clase detrás de un puntero opaco, reduciendo los tiempos de compilación al evitar `#include` innecesarios en la cabecera.

GUID / IID

Globally Unique Identifier / *Interface Identifier*. Número entero de 128 bits utilizado en sistemas Microsoft para identificar de manera única clases e interfaces COM.

Forward Declaration

Declaración previa de un tipo o estructura antes de definirlo formalmente, lo que permite utilizar punteros o referencias al tipo sin incluir su archivo de cabecera completo.

RenderDoc

Herramienta de depuración independiente que permite capturar un fotograma individual de una aplicación gráfica para inspeccionar cada llamada de dibujo (*Draw Call*), búferes y estados del pipeline.

Concatenar

Operación lógica de unir dos o más cadenas de caracteres, arreglos o secuencias de datos secuencialmente una detrás de otra para formar un único bloque continuo.

Metaprogramación

Técnica de programación informática en la que se escriben programas que escriben, modifican, o analizan otros programas (o a si mismos como si fueran datos.)